



Universidade de Brasília
Departamento de Ciência da Computação

Fila

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Definição

- ▶ As filas são implementadas e se comportam de modo muito similar às listas, consideradas, muitas vezes, como um tipo especial de lista em que a inserção e a remoção são realizadas sempre em extremidades distintas;
- ▶ Desse modo, se quisermos acessar um determinado elemento da fila, devemos remover todos os que estiverem à frente dele;
- ▶ São conhecidas como estruturas do tipo **primeiro a entrar, primeiro a sair** ou **FIFO** (*First In First Out*);



Tipos de pilhas

- ▶ Basicamente, existem dois tipos de implementações principais para uma pilha:
 - ✓ Alocação estática com acesso sequencial: o espaço de memória é alocada no momento da compilação do programa, ou seja, é necessário definir o número máximo de elementos que a fila irá possuir. Desse modo, os elementos são armazenados de forma consecutiva na memória (como em um *array* ou vetor) e a posição de um elemento pode ser facilmente obtida a partir do início da fila;
 - ✓ Alocação dinâmica com acesso encadeado: o espaço de memória é alocado em tempo de execução, ou seja, a fila cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos. Nessa implementação, cada elemento pode estar em uma área distinta da memória, não necessariamente consecutivas. É necessário então que cada elemento da fila armazene, além da sua informação, o endereço de memória onde se encontra o próximo elemento. Para acessar um elemento, é preciso percorrer todos os seus antecessores na fila.

Tipo Abstrato de Dados

- ▶ Independentemente do tipo de alocação e acesso usado na implementação de uma fila, as seguintes operações básicas são sempre possíveis:
 - ✓ **queue()**: cria a fila
 - ✓ **enqueue(item)**: insere um novo item na fila;
 - ✓ **dequeue()**: remove o item do início da fila;
 - ✓ **front()**: retorna o item no início da fila;
 - ✓ **isEmpty()**: testa se a fila está vazia;
 - ✓ **size()**: retorna o número de itens na fila.

Implementação

► Exemplo:

- ✓ Queue(4) - 100
- ✓ Queue(3) - 250
- ✓ Queue(7) - 300
- ✓ Dequeue()
- ✓ Queue(6) - 130
- ✓ Queue(5) - 195

End	Item	Prox
195	3	NULL
130	6	195
300	7	130
250	3	300
100	4	250

ini	fim
100	250

```
void queue(Fila* f, int v){
```

```
    FilaNo* n = (FilaNo*)malloc(sizeof(FilaNo));
    n->item = v;           /* armazena informação */
    n->prox = NULL;        /* novo nó passa a ser o último */
    if (f->fim != NULL) { /* verifica se fila não está vazia */
        f->fim->prox = n;
    } else {               /* fila está vazia */
        f->ini = n;
    }
    f->fim = n;            /* fila aponta para novo elemento */
}
```

```
float dequeue(Fila* f){
```

```
    FilaNo* t = f->ini;
    int v = t->item;
    f->ini = t->prox;
    if (f->ini == NULL) { /*verifica se fila ficou vazia */
        f->fim = NULL;
    }
    free(t);
    return v;
}
```

Desafio: cadeia palíndroma

- ▶ Uma cadeia é palíndroma se ela é igual à sua inversa (ignorando-se os espaços). Por exemplo, *"ovo"*, *"subi no onibus"*, *"a sacada da casa"* e *"anotaram a data da maratona"* são cadeias palíndromas;
- ▶ Para obter a cadeia direta, sem os espaços, basta percorrer a cadeia original, da esquerda para a direita, inserindo numa fila cada letra encontrada.
- ▶ Quando essas letras forem removidas da fila, elas formarão uma cadeia na mesma ordem da cadeia original.
- ▶ Analogamente, para obter a cadeia inversa, basta percorrer a cadeia original, da esquerda para a direita, inserindo as letras numa pilha.
- ▶ Quando as letras forem removidas da pilha, elas formarão uma cadeia na ordem inversa da cadeia original.
- ▶ Portanto, comparando-se as letras removidas da fila e da pilha, podemos determinar se a cadeia original é ou não palíndroma.

Desafio: cadeia palíndroma

